

Software Requirements Specification (SRS)

Task Manager

Version: 1.0

Project Type: Python Mini Project 6

Frontend: HTML/CSS

Backend: Python (Flask)

Storage: JSON File

Document Control

Prepared by: Md. Anjir Jaman

Date: 05-09-2026

Version: 1.0 – Initial draft

1. Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Intended User

2. Overall Description

- 2.1 Product Perspective
- 2.2 Main Features
- 2.3 Operating Environment

3. Functional Requirements

4. Data Requirements

5. User Interface Requirements

- 5.1 Task List Page
- 5.2 Add Task Form
- 5.3 Edit Task Page
- 5.4 Filter Controls

6. Backend Requirements

7. Data Storage Requirements

8. Non-Functional Requirements

9. Error Handling

10. System Constraints

11. Assumptions

12. Acceptance Criteria

13. Project Schedule and Milestones

14. Requirement Summary

1. Introduction

1.1 Purpose

The Task Manager is a web-based application that allows a user to create, edit, update, and delete personal or work tasks.

The application will provide a simple HTML-based interface through which users can add tasks, edit task details, quickly update task status, filter tasks, and delete tasks.

Task records will be stored in a local JSON file so that the data remains available after the application is closed.

1.2 Scope

The system will provide the following main functions:

- Add a task with a title, description, deadline, and priority
- View all tasks in a list
- Edit an existing task's details
- Quickly update a task's status (Pending, In Progress, Completed)
- Filter tasks by their current status
- Delete a task
- Store task records in a local JSON file
- Load existing tasks when the application starts
- Validate user input

The front-end will be developed using basic HTML/CSS, while Python (Flask) will handle the application logic and data processing.

1.3 Intended User

The system is intended for a user who wants to organize and track a personal list of tasks. The user does not need advanced technical knowledge to operate the application.

2. Overall Description

2.1 Product Perspective

The Task Manager will operate as a locally-hosted Python web application, served using Flask and accessed through a web browser at localhost. The system will use a local JSON file for data storage. No external database or internet connection is required.

Basic system flow:

User → HTML/CSS Front End → Python (Flask) Backend → JSON file (tasks.json)

Figure 1: System Architecture

The frontend will collect user input and display information across two pages: a task list page and an edit-task page. The Python backend will process requests, perform validation, manage task records, and handle file operations. The JSON file will provide persistent storage.

2.2 Main Features

- Add Task
- View Task List
- Edit Task
- Update Status
- Filter by Status
- Delete Task

2.3 Operating Environment

The application will require:

- A computer
- Python 3 with Flask installed
- A modern web browser
- A Python development environment such as VS Code or PyCharm
- Local file storage

3. Functional Requirements

FR-01: Add Task

The system shall provide a form for adding a new task with Title, Description, Deadline, and Priority.

The system shall require Title and Deadline before a task can be added.

The system shall assign a new task a default status of Pending.

On success, the system shall save the new task and refresh the task list.

FR-02: View Task List

The system shall display all tasks in a list, showing each task's title, deadline, priority, and status.

If no tasks exist, the system shall display an appropriate message.

FR-03: Edit Task

The system shall provide a separate edit page pre-filled with a task's existing details.

The user shall be able to update the title, description, deadline, priority, and status of the task.

The system shall validate and save the updated task, then return the user to the task list.

FR-04: Update Status

The system shall allow the user to change a task's status directly from the task list, without opening the full edit page.

The system shall accept only Pending, In Progress, or Completed as valid status values.

FR-05: Filter Tasks

The system shall allow the user to filter the task list by status (All, Pending, In Progress, or Completed).

The system shall update the displayed list to match the selected filter.

FR-06: Delete Task

The system shall allow the user to delete an existing task.

The system shall remove the task from storage and refresh the displayed list.

FR-07: Save Tasks

The system shall save the task list to the local JSON file after every add, edit, status update, or delete operation.

FR-08: Load Tasks

The system shall load existing tasks from the JSON file when the application starts.

If the JSON file does not exist, the system shall create it automatically and start with an empty task list.

FR-09: Input Validation

The system shall reject a task submission with a missing Title or Deadline.

The system shall accept only a valid Priority value (High, Medium, or Low).

The system shall accept only a valid Status value (Pending, In Progress, or Completed).

The system shall display clear error messages when invalid information is submitted.

4. Data Requirements

Each task record shall contain:

Field	Description	Required
ID	Unique task identifier	Yes
Title	Short name of the task	Yes
Description	Additional detail about the task	Optional
Deadline	Due date for the task, in YYYY-MM-DD format	Yes
Priority	One of: High, Medium, Low	Yes
Status	One of: Pending, In Progress, Completed	Yes

5. User Interface Requirements

The application shall use a basic HTML/CSS frontend, structured across a task list page and a separate edit-task page.

5.1 Task List Page

The page shall display all tasks, each showing its title, deadline, priority, and status.

Each task shall provide an Edit link, a Delete button, and a quick status control.

5.2 Add Task Form

The task list page shall include a form with fields for:

- Title
- Description
- Deadline
- Priority

It shall provide an Add Task button.

5.3 Edit Task Page

The page shall display the same fields as the Add Task form, pre-filled with the selected task's current values, plus its Status.

It shall provide a Save Changes button and a link back to the task list.

5.4 Filter Controls

The task list page shall provide a control (such as a dropdown or a set of links) to filter tasks by status.

6. Backend Requirements

The Python (Flask) backend shall:

- Serve the task list page and the edit-task page
- Receive and validate new task submissions
- Receive and validate task edit submissions
- Update a task's status on request
- Delete a task on request
- Filter tasks by status when requested
- Read from and write to the JSON storage file
- Return appropriate results and messages to the front end

7. Data Storage Requirements

The system shall use JSON for local data storage.

Example tasks.json:

```
[
  {"id": "1", "title": "Write report", "description": "", "deadline": "2026-09-10", "priority": "High", "status": "Pending"}
]
```

The system shall support:

- Creating task records
- Reading task records
- Updating task records
- Deleting task records
- Persistent storage across sessions

8. Non-Functional Requirements

NFR-01: Usability

The interface should be simple, clear, and easy to navigate.

NFR-02: Performance

Normal operations should complete within a reasonable time for the expected small number of tasks.

NFR-03: Reliability

The system should correctly save and retrieve task information during normal operation.

NFR-04: Maintainability

The Python code should be organized clearly, separating routing, validation, and file handling, so the application can be modified or extended easily.

NFR-05: Compatibility

The frontend should work with a modern web browser.

NFR-06: Data Persistence

Task information shall remain available after the application is closed and reopened.

9. Error Handling

The system shall handle common errors without crashing.

The system shall provide appropriate messages for:

- Missing Title or Deadline when adding or editing a task
- An invalid Priority or Status value
- Editing or deleting a task that no longer exists
- A missing or corrupted tasks.json file
- An empty task list

After an error, the user should be able to correct the input and try again without restarting the application.

10. System Constraints

- The backend will be developed using Python and Flask.
- The frontend will use basic HTML/CSS, with no JavaScript framework.
- Data will be stored locally using JSON.
- No internet connection is required for normal operation.
- No external database is required.
- The system is intended for a small number of tasks.
- The application will run locally through a web browser.
- The project will remain within the scope of a Python mini project.

11. Assumptions

- Python 3 and Flask are installed on the development computer.
- A modern web browser is available.
- The user has permission to read and write files in the project directory.
- The tasks.json file is accessible to the application.

12. Acceptance Criteria

The project will be considered complete when:

- The HTML frontend loads successfully.
- A task can be added with a title and deadline.
- Tasks can be viewed in a list.
- A task's details can be edited and saved.
- A task's status can be updated directly from the list.
- The task list can be filtered by status.
- A task can be deleted.
- Tasks are saved to the JSON file.
- Existing tasks load correctly when the application restarts.
- Invalid input is handled correctly without crashing.
- Appropriate success and error messages are displayed.
- All major functions pass their test cases.

13. Project Schedule and Milestones

The project is scheduled to be completed in a single working session (approximately 6 hours), broken down by task as shown in Figure 2. The schedule follows the SDLC sequence defined in this document: requirements review, design, implementation, testing, and reporting.

Task	9am	10am	11am	12pm	1pm	2pm
Set up Flask project, JSON load/save helpers						
Task list page + Add Task form/route						
Edit Task page + update route						
Update Status + Delete route						
Filter by status						
Testing + write report						

Figure 2: Development schedule

14. Requirement Summary

FR-01	Add task	FR-06	Delete task
FR-02	View task list	FR-07	Save tasks
FR-03	Edit task	FR-08	Load tasks
FR-04	Update status	FR-09	Validate input
FR-05	Filter tasks	NFR-01	Usability
NFR-02	Performance	NFR-05	Compatibility
NFR-03	Reliability	NFR-06	Data persistence
NFR-04	Maintainability		